

系统开发工具基础

第 3 周课后实验报告

姓名：彭俊皓 学号：25020007100

2026 年 9 月 13 日

仓库：<https://github.com/pjh456/sys-devtool/>

本报告覆盖第 3 周课堂四道实验（week3/q09–q12）的操作过程与结果，以及 missing-semester 三部分课后练习：《命令行环境》的任务控制与别名（homework/command-line/signal、aliases）、《Shell 工具和脚本》的最近修改文件（homework/command-line/file），以及《命令行环境》的配置文件项目 dotfiles（homework/command-line/aliases/3-8，以 CI 作为新系统安装验证依据）；全部结果均在本机重新执行验证。

表 1: 第 3 周课后实验完成情况

部分	内容	关键工具	状态
一	课堂实验回顾（q09–q12）	build、venv、pytest、pdb、torch	🟢 完成
二	课后：《命令行环境》任务控制与别名	jobs、pgrep、pkill、alias	🟢 完成
三	课后：《Shell 工具和脚本》最近修改文件	find、sort、head、cut	🟡 完成
四	课后：《命令行环境》配置文件（dotfiles）	git、install.sh、GitHub Actions	🟡 完成

一

课堂实验回顾（q09–q12）

q09

从源码构建并在干净环境安装 Wheel

📖 题目要求

- 创建 src/greetlab/__init__.py、给定的 cli.py 与 pyproject.toml，并把“学号”替换为自己的学号；
- 在已安装 build 工具的课程环境中运行 python -m build 生成 wheel；
- 新建一个干净虚拟环境，只从生成的 wheel 安装；
- 切换到 q09 之外运行 sdt-greet --name < 学号 >，确认输出正确。

>_ 操作过程

```

1 .venv/bin/python -m build          # 生成 sdist 与 wheel
2 # ... Successfully built greetlab_25020007100-0.1.0.tar.gz and
3 #   greetlab_25020007100-0.1.0-py3-none-any.whl
4 python3 -m venv /tmp/opencode/q09-clean-venv      # 干净虚拟环境
5 /tmp/opencode/q09-clean-venv/bin/pip install --no-index \
6   dist/greetlab_25020007100-0.1.0-py3-none-any.whl  # 只装本地 wheel
7 cd /tmp/opencode                                  # 切到 q09 之外
8 /tmp/opencode/q09-clean-venv/bin/sdt-greet --name 25020007100

```

✓ 结果与验证

```

$ /tmp/opencode/q09-clean-venv/bin/pip install --no-index ...
Successfully installed greetlab-25020007100-0.1.0
$ cd /tmp/opencode && .../sdt-greet --name 25020007100
Hello, 25020007100!
$ .../python -c "import greetlab.cli as c; print(c.__file__)"
/tmp/opencode/q09-clean-venv/lib/python3.14/site-packages/greetlab/cli.py

```

wheel 内含 greetlab 包的 `__init__.py`、`cli.py` 与 `dist-info` 元数据。`greetlab.cli` 解析到干净 `venv` 的 `site-packages` (而非 `src/`)，证明是“只从 wheel 安装”而非 `editable` 安装。

💡 要点与注意事项

`pip install --no-index` 禁止联网，确保依赖只能来自本地 wheel；构建产物 `dist/` 与虚拟环境均在 `.gitignore` 中，不入库。初次提交漏了 `__init__.py`：缺它时 `setuptools` 把 `greetlab` 当命名空间包 (`greetlab.__file__` 为 `None`)，虽能运行却不满足题目“创建 `__init__.py`”；补齐后 wheel 才真正包含该文件。另外，开发用的 `.venv` 是 `editable` 安装，`import` 指回 `src/`，不能作为“装了 wheel”的依据，必须另建独立 `venv` 验证。

q10 让编程智能体进入可验证的修复循环

📖 题目要求

- 复制 q09 为 q10，添加一个失败测试：`name` 全为空白字符时，`main` 应以 `SystemExit(2)` 退出；
- 先运行测试确认失败，再向智能体说明目标、约束和测试命令，要求它修改实现并运行测试；
- 人工检查 diff，撤销无关修改，并再次运行测试；
- 在 `ai_log.md` 中用不超过 5 行记录核心提示、智能体改动和人工验证。

🔗 失败测试 (q10/tests/test_cli.py)

```

1 import pytest
2 from greetlab.cli import main
3
4 def test_blank_name(monkeypatch, capsys):
5     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", " "])
6     with pytest.raises(SystemExit) as exc:
7         main()
8     assert exc.value.code == 2

```

智能体改动 (src/greetlab/cli.py, 仅+2行)

```

1     a = p.parse_args()
2 +     if not a.name.strip():
3 +         p.error("argument --name: value must not be blank")
4     print(f"Hello, {a.name}!")

```

结果与验证

```

$ .venv/bin/python -m pytest tests -q
1 passed in 0.01s

$ .venv/bin/sdt-greet --name " "
usage: sdt-greet [-h] --name NAME
sdt-greet: error: argument --name: value must not be blank
$ echo $?
2
$ .venv/bin/sdt-greet --name alice
Hello, alice!

```

ai_log.md (3行): 核心提示要求 --name 全空白时以 SystemExit(2) 退出并用仓库虚拟环境跑 pytest; 智能体只改 cli.py 2 行; 人工审查 diff 无无关修改后复跑通过。

要点与注意事项

argparse 的 p.error() 会向 stderr 打印用法错误并以 SystemExit(2) 退出, 与参数解析错误保持一致; “先红后绿”是智能体修复循环的关键——先在未修改的实现上跑出失败, 再让智能体改到测试通过, 最后人工复核 diff, 避免它顺手改动无关文件。

q11 把“无法处理”的协作材料改成可执行信息

题目要求

- 重写 Issue: 包含环境、复现命令、期望结果和实际结果, 未知信息写“待确认”;
- 重写提交信息: 标题使用祈使语气, 正文说明问题和解决方案;

- 重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking / Suggestion / Nit;
- 全文控制在 400 字以内，保持专业、简洁。

🔗 改写结果 (q11/communication.md)

```
# Issue
- 环境: Linux, Python 3.12, greetlab 0.1.0 (Windows 影响待确认)
- 复现命令: `sdt-greet --name " "`
- 期望结果: 报告 --name 用法错误, 退出码 2
- 实际结果: 输出 `Hello, !`, 退出码 0
- 备注: 原 Issue “Windows 上运行不了” 无复现记录, 待确认

# 提交信息
fix: 拒绝仅含空白字符的 --name 值
problem: 纯空白姓名被当作合法输入, 正常打印问候且退出码 0, 调用方无法发现错误。
solution: 解析后用 strip() 校验, 空白时以 p.error() 报错并退出 2。

# 评审意见
[Blocking]
- 行为: name 全为空白时 main 仍打印问候语并以 0 退出
- 风险: 上游依赖退出码判断输入合法性, 空白姓名会被静默接受
- 建议: 增加空白校验并以 2 退出, 并补充回归测试
```

✅ 结果与验证

改写后非空白字符 363 字 < 400; Issue 补齐了环境、复现、期望、实际四要素, 无法核实的 Windows 影响标注为“待确认”; 提交信息标题为祈使句, 正文分 problem / solution; 评审意见按“行为—风险—建议”组织并标注 [Blocking]。

💡 要点与注意事项

协作材料的价值在于“可执行”: Issue 必须能被他人照着复现, 所以把已知事实(命令、退出码、实际输出)写全, 未知信息显式标“待确认”, 而不是含糊描述。

q12 修复一个可复现的线性回归训练循环

📖 题目要求

- 补全训练循环, 正确调用 zero_grad、backward 和 step;
- 训练后调用 model.eval(), 并在 torch.no_grad() 中计算最终损失;
- 打印最终损失、weight 和 bias; 最终损失应小于 0.001;
- 不得直接把 weight 和 bias 赋值为 3 和 -1。

</> 补全后的关键片段 (q12/train.py)

```

1 for _ in range(200):
2     pred = model(x)
3     loss = loss_fn(pred, y)
4     opt.zero_grad()          # 清空梯度
5     loss.backward()          # 反向传播
6     opt.step()               # 更新参数
7
8 model.eval()                 # 评估模式
9 with torch.no_grad():        # 关闭梯度记录
10     final_loss = loss_fn(model(x), y)
11     print(f"final loss: {final_loss.item():.6f}")
12     print(f"weight: {model.weight.item():.6f}")
13     print(f"bias: {model.bias.item():.6f}")

```

✓ 结果与验证

```

$ .venv/bin/python train.py
final loss: 0.000000
weight: 2.999997
bias: -1.000000

```

最终损失约 0 (< 0.001), $\text{weight} \approx 3$ 、 $\text{bias} \approx -1$, 与真实关系 $y = 3x - 1$ 吻合, 且这两个值由训练得到、未手工赋值。

💡 要点与注意事项

三步顺序固定: 先 `zero_grad()` 清掉上一轮梯度, 再 `backward()` 反传求导, 最后 `step()` 更新参数; 评估阶段用 `model.eval()` 切换模式并在 `no_grad()` 下计算, 避免多余的计算图与梯度开销。

二 课后练习: missing-semester 《命令行环境》——任务控制与别名**任务控制·1 用 pgrep/pkill 结束进程而无须手抄 PID****📖 题目要求**

在终端执行 `sleep 10000`, 用 `Ctrl-Z` 切到后台并用 `bg` 继续运行; 再用 `pgrep` 查找其 `pid`、用 `pkill` 结束进程, 全程不手动输入 `pid`。(提示: 使用 `-af` 标记)

</> 解答 (signal/1.sh)

```
1 #!/bin/sh
2 set -euo pipefail
3 set -m
4
5 sleep 10000 &
6 SLEEP_PID=$!
7
8 sleep 0.5
9 kill -TSTP $SLEEP_PID      # SIGTSTP, 等价于 Ctrl-Z
10 sleep 0.5
11
12 bg
13 pgrep -af "sleep 10000"
14 pkill -f "sleep 10000"
```

✓ 结果与验证

```
$ sh signal/1.sh
[1] sleep 10000 &
2297212 sleep 10000
```

pgrep -af 输出 PID + 完整命令行; pkill -f 按同一模式结束该进程, 过程中没有出现手工抄写的 PID, 脚本退出码为 0。

💡 要点与注意事项

kill -TSTP 发出终端停止信号 (等价于 Ctrl-Z), 配合 set -m (开启作业控制) 与 bg 让任务转到后台继续; pgrep -af / pkill -f 匹配整条命令行, 比只匹配进程名更适合 sleep 10000 这类“进程名相同、参数不同”的场景。

任务控制·2 pidwait: 等待任意进程结束

📖 题目要求

wait 只能等待子进程, 换个会话就失效。编写 bash 函数 pidwait: 接受一个 pid 作为参数, 一直等待直到该进程结束; 用 sleep 避免浪费 CPU。(提示: kill -0 不发送信号, 但进程不存在时返回非 0)

🔗 解答 (signal/2.sh)

```
1 pidwait() {
2     if [ $# -lt 1 ]; then
3         echo "Usage: pidwait <pid>" >&2
4         return 1
5     fi
6 }
```

```
7     local PID=$1
8     while kill -0 $PID 2>/dev/null; do
9         sleep 1
10    done
11 }
```

✓ 结果与验证

```
$ source signal/2.sh
$ sleep 3 & p=$!
$ date +%T; pidwait $p; date +%T
16:58:36
16:58:39
```

pidwait 在 sleep 3 结束后立即返回，时间差约 3 秒。

💡 要点与注意事项

kill -0 只做存在性探测、不真正发信号：进程在返回 0，进程不存在返回非 0，因此可用作循环条件；每轮 sleep 1 轮询，避免忙等占满 CPU。函数以 source 载入当前 shell 后才可用。

别名·1 把误输入的 dc 变成 cd

📖 题目要求

创建一个 dc 别名，其功能是当我们把 cd 错误输入成 dc 时也能正确执行。

🔗 解答 (aliases/1.sh)

```
1 alias dc=cd
```

✓ 结果与验证

```
$ source aliases/1.sh
$ alias dc
alias dc='cd'
```

💡 要点与注意事项

别名在交互式 shell 中才生效，且默认不持久；要长期可用需写入启动文件（如 ~/.bashrc）并重新加载。演示脚本须 source 进当前 shell。

别名·2 统计最常用命令并建立别名

📋 题目要求

执行讲义给出的 `history` 统计管道（见下），获取最常用的十条命令，并尝试为它们创建别名。

🔗 解答 (`aliases/2.sh`)

```
1 history | awk '{ $1="" ; print substr($0,2) }' | sort | uniq -c | sort -n | tail -n 10
2
3 alias ll='ls -lah'
4 alias la='ls -A'
5 alias gs='git status'
6 alias ga='git add'
7 alias gc='git commit'
8 alias gp='git push'
9 alias gd='git diff'
10 alias grep='rg --color=auto'
11 alias cat='bat --paging=never'
12 alias py='python3'
```

✅ 结果与验证

```
$ source aliases/2.sh
$ alias | grep -E '^(alias (ll|la|gs|ga|gc|gp|gd|grep|cat|py))'
alias cat='bat --paging=never'
alias ga='git add'
alias gc='git commit'
alias gd='git diff'
alias gp='git push'
alias grep='rg --color=auto'
alias gs='git status'
alias la='ls -A'
alias ll='ls -lah'
alias py='python3'
```

💡 要点与注意事项

`history` 是交互式 shell 的内建命令；实际高频命令（`git`、`ls`、`grep` 等）按日常使用整理成上述别名。计数前必须先 `sort`，因为 `uniq -c` 只统计相邻的重复行。

三 课后练习：missing-semester 《Shell 工具和脚本》——最近修改的文件

文件·1 递归查找最近修改的一个文件

📖 题目要求

编写一个命令或脚本, 递归地查找文件夹中最近修改的文件。(提示: 组合 `find`、`sort`、`head`, 并留意文件名中的空格)

🔗 解答 (file/1.sh)

```
1 #!/bin/sh
2 if [ $# -lt 1 ]; then
3     echo "Directory is required!" >&2
4     exit 1
5 fi
6 FP=$1
7 if [ ! -e $FP ]; then
8     echo "Directory is missing!" >&2
9     exit 1
10 fi
11 find $FP -type f -printf '%T@ %p\0' | sort -rnz | head -z -n 1 | cut -z -d' ' -f2-
```

✅ 结果与验证

```
$ sh file/1.sh homework/course-shell
homework/course-shell/18/check_command.sh
```

输出为 `course-shell` 下 `mtime` 最新的文件, 路径完整、无附加时间戳。

💡 要点与注意事项

`find -printf '%T@ %p\0'` 以 NUL 分隔输出 “mtime 时间戳 + 路径”; `sort -rnz` 按时间戳倒序 (-z 以 NUL 分列), `head -z -n1` 取第一个, `cut -z -d' ' -f2-` 去掉时间戳只留路径。全程 NUL 分隔, 文件名含空格或换行都不会错位。

文件·2 按修改时间列出全部文件

📖 题目要求

更通用的做法: 按最近的修改时间列出文件夹中的文件 (可以截断为若干条)。

🔗 解答 (file/2.sh)

```
1 # 前置校验同 file/1.sh
2 find $FP -type f -printf '%T@ %p\0' | sort -rnz | cut -z -d' ' -f2- | tr '\0' '\n'
```

结果与验证

```
$ sh file/2.sh homework/course-shell | head -3
homework/course-shell/18/check_command.sh
homework/course-shell/18/explain.txt
homework/course-shell/17.sh
```

要点与注意事项

与题 1 同源，只是去掉了 `head -z -n1`，得到按 `mtime` 倒序的全量列表；末尾 `tr '\0' '\n'` 把 NUL 分隔转回换行，便于逐行阅读。

四 课后练习：《命令行环境》配置文件（dotfiles）与 CI 验证

配置文件·1-6 dotfiles 项目：版本控制、安装脚本与新系统验证

题目要求

1. 为配置文件新建一个文件夹，并设置好版本控制；
2. 在其中添加至少一个配置文件（如 `shell`），包含一些自定义设置；
3. 建立一种在新设备快速安装配置的方法（如 `ln -s` 或专用工具）；
4. 在新的虚拟机上测试该安装脚本；
5. 将现有配置文件移入项目仓库；
6. 将项目发布到 GitHub。

</> 解答（submodule aliases/3-8）

- **版本控制与发布**：项目以独立 Git 仓库管理，发布在 [pjh456/dotfiles](#)，并以 submodule 形式挂到本仓库 `aliases/3-8`；
- **配置文件**：`etc/` 下含 `.bashrc`、`.bash_profile` 及 Hyprland 桌面的 `hypr`、`waybar`、`swaync`、`systemd/user` 等配置；
- **安装脚本**：`install.sh` 把 `etc/` 部署到 `$HOME` 并先备份原件，支持 `--packages`、`--restore` 等选项，且幂等（重复运行安全）。

结果与验证

“在新系统测试安装脚本”由 GitHub Actions 在**全新环境**中实测：

```
arch-install      success  1m37s   # archlinux 容器里真实 ./install.sh --packages
```

```

debian-install  success  2m1s    # debian:testing 容器里真实安装
install-dry-run success  5s      # 全新 HOME 上验证部署/备份/幂等/--restore
shellcheck      success  15s    # 脚本静态检查
json            success  3s      # waybar/swaync JSON 校验

```

当前 HEAD (f6e0156) CI 全绿，run 链接如下。

<https://github.com/pjh456/dotfiles/actions/runs/34182687378>

ubuntu-latest runner 每次都是一款全新 VM，Arch/Debian job 又在其上起干净的官方容器，因此可复现地充当“新虚拟机测试安装脚本”的依据。

💡 要点与注意事项

install.sh 采用“复制 + 部署清单 + 备份”而非单纯 `ln -s`，便于 `--restore` 回滚；CI 用真实的 `install.sh --packages`（而非模拟）覆盖 Arch 与 Debian 两种发行版。局限：CI 只验证安装/部署逻辑，不在容器中启动 Hyprland/waybar 等桌面会话，GUI 运行时行为未覆盖。

五 总结与收获

• 课堂实验 (q09–q12)

- 打包的最小闭环是“源码 + `pyproject.toml` → `python -m build` → 干净 `venv` 只装 `wheel` → 换目录运行”；缺 `__init__.py` 会退化成命名空间包，`editable` 安装不能代替 `wheel` 安装来验证。
- 智能体修复要“先红后绿 + 人工复核 diff”，把测试命令作为约束交给它。
- 协作材料要写成可执行信息：Issue 可复现、提交信息分问题/方案、评审标注严重级别。
- PyTorch 训练循环顺序为 `zero_grad`、`backward`、`step`，评估用 `eval()` 搭配 `no_grad()`。

• 课后练习 (missing-semester)

- 作业控制：Ctrl-Z 发 `SIGTSTP`，`bg` 转后台，`pgrep -af/pkill -f` 按命令行定位进程；`kill -0` 可做进程存在性探测，用来实现 `pidwait`。
- 别名默认不持久，需写入启动文件；`uniq -c` 前必须 `sort`。
- 用 `find -printf '%T@ %p\n'` 搭配 `sort -z/cut -z` 可按 `mtime` 排序，且安全处理含空格文件名。
- 配置文件项目用“复制 + 备份 + 清单”的 `install.sh` 管理，Git + CI（真实 `install --packages`）构成可复现的安装验证链。